ICT Express 12 (2026) 301–305

Contents lists available at ScienceDirect

ICT Express

journal homepage: www.elsevier.com/locate/icte

Learning graph based individual intrinsic reward for multi-agent reinforcement learning

Seokhun Ju [a], Seungyub Han [a], Taehyun Cho [a], Jungwoo Lee [a,*], Taeyoung Lee [b], Minkyoung Kim [b], Jinho An [b]

[a] Department of Electrical and Computer Engineering, Seoul National University, Seoul, South Korea
[b] Intelligent SW team, Hanwha System, Seongnam, South Korea

ARTICLE INFO

Keywords:
Multi-agent reinforcement learning
Preference based reinforcement learning
Graph neural network
Intrinsic reward

ABSTRACT

Designing a reward function is a critical challenge in reinforcement learning. However, as environments become more complex and tasks grow more difficult, designing a reward function that drives optimal behavior becomes increasingly challenging. To overcome these issues, Preference based reinforcement learning has proposed methods that learn reward functions based on the preference between two trajectories, thereby eliminating the need for handcrafted reward function. In multi-agent reinforcement learning, the challenge is even greater due to the complex interactions among agents, which makes designing a single global reward function even more difficult. In this paper, we show that when a single global reward function is learned via preference-based reinforcement learning in multi-agent setting, it often fails to capture sufficient information for optimal policy learning. Instead, we propose a method for learning individual reward functions that provide additional guidance for each agent's optimal policy. Our approach, which leverages graph structures and preference-based reinforcement learning, outperforms the method based on learning a single, global reward function.

1. Introduction

Reinforcement learning [1] is a method that aims to learn policies that maximize the cumulative expected reward. It has shown excellent performance in tasks that require sequential decision-making [2,3] and has been successfully applied in various domains [4–6]. In most cases, reward functions are manually designed by humans, and it is assumed that these hand-designed functions will guide the agent toward an optimal policy. However, as the environment and objectives become more complex, designing an appropriate reward function becomes increasingly challenging, often requiring significant time and resources. Moreover, policies trained to maximize the designed reward often exhibit unintended behaviors or exploit the reward function by repeating certain actions — a phenomenon known as the Reward Exploitation Problem — which further complicates reward design. To address these challenges, preference-based reinforcement learning has been proposed. Instead of using a handcrafted reward, this approach learns a reward function from trajectory segments and human preference between them. The resulting reward function is then integrated into reinforcement learning algorithms to train the agent's policy. The underlying assumption is that trajectories with higher cumulative rewards are preferred, and artificial neural networks are employed to model the reward function.

In cooperative multi-agent reinforcement learning, where agent interactions must be considered, the complexity of the reward function increases, making its design challenging. Multi-agent reinforcement learning methods that learn individual agent policies from a single global reward function address the credit assignment problem by estimating each agent's contribution to the global reward [7]. However, while a global reward function is easier to design, it often fails to provide the detailed information needed to guide each agent toward its optimal actions. This paper identifies the limitations of preference-based reinforcement learning in multi-agent reinforcement learning when a single global reward function is learned and used to train policies through multi-agent reinforcement learning algorithms This paper identifies the limitations of using a single learned global reward in preference-based multi-agent reinforcement learning algorithms [8,9]. To address these limitations, we propose a method learning individual reward functions that can provide additional information about each agent's optimal policy. By leveraging a graph structure to model interactions among agents, the proposed method learns individual reward

* Corresponding author.
E-mail addresses: seokhunju@cml.snu.ac.kr (S. Ju), syhan@cml.snu.ac.kr (S. Han), talium@cml.snu.ac.kr (T. Cho), junglee@snu.ac.kr (J. Lee), ty.lee@hanwha.com (T. Lee), kmk0224@hanwha.com (M. Kim), babupeople@hanwha.com (J. An).

https://doi.org/10.1016/j.icte.2025.07.010
Received 24 April 2025; Received in revised form 9 July 2025; Accepted 29 July 2025
Available online 20 August 2025
2405-9595/© 2025 The Authors. Published by Elsevier B.V. on behalf of The Korean Institute of Communications and Information Sciences. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

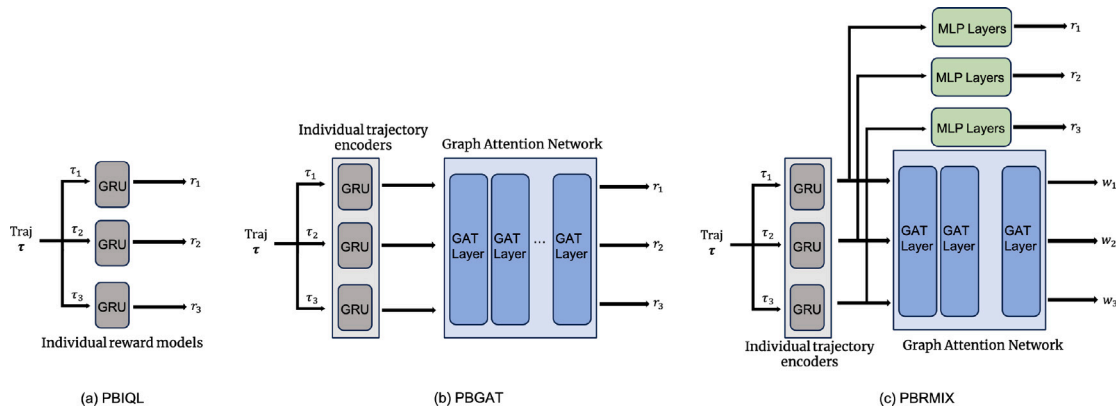


Fig. 1. Illustration of three proposed methods, (a) PBIQL, (b) PBGAT, (c) PBRMIX.

functions that deliver concrete information about optimal policies for each agent. Experimental results confirm that the proposed method outperforms approaches that rely on a learned single global reward function for multi-agent reinforcement learning.

2. Background

2.1. Multi-agent reinforcement learning

Cooperative multi-agent reinforcement learning is defined as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) [10]. A Dec-POMDP is represented as $G = (S, U, P, A, r, Z, O, n, \gamma)$. Here, $s \in S$ denotes the state, and at each step, each agent $a \in A$ takes an action $u \in U$. The state transition model of the environment is given by $P(s'|s, u)$, where u is the joint action of all agents, and $r(s, u)$ represents the global single reward function shared among all agents. Since this is a partially observable Markov decision process (POMDP), the observation space Z is defined, with each observation modeled as $O(s, a) : S \times A \rightarrow Z$. Additionally, n represents the number of agents, and γ denotes the discount factor.

Widely recognized algorithms in multi-agent reinforcement learning include VDN [8] and QMIX [9]. VDN and QMIX are value-based reinforcement learning algorithms that learn the value function using the Bellman equation, under the assumption that the global state–action value function is the sum — or a positive weighted sum — of the individual state–action value functions for each agent.

As mentioned in the introduction, designing a global single reward function that provides sufficient information about the optimal actions of each agent in complex multi-agent environments is challenging. Additionally, effectively performing credit assignment based on the given global reward function is difficult. To address these issues, research has focused on defining individual reward functions for efficient multi-agent policy learning [11]. The experimental results of these studies demonstrate that introducing appropriate individual reward functions in multi-agent environments can improve learning performance and stabilize the training process.

2.2. Preference based reinforcement learning

Preference-based reinforcement learning directly learns a reward function using preference label between two given trajectories. A trajectory is defined as $\tau = (s_0, a_0, \ldots, s_T, a_T)$, and a segment of the trajectory is represented as $\sigma = (s_k, a_k, \ldots, s_{k+h-1}, a_{k+h-1})$. The data consists of trajectory segments σ^0, σ^1 and a preference label $y \in \{(1, 0), (0, 1)\}$, defined as (σ^0, σ^1, y). To model the relationship between the reward function $r_\psi(s, a)$ to be learned, and the trajectory segments, we define the probability of preferring σ^1 over σ^0 with the sum of reward values $\sum_t r_\psi(s_t, a_t)$ and the Bradley–Terry model (Eq. (1)), and train the reward model by comparing it with the data labels y using cross entropy loss (Eq. (2)). Preference labels can be assigned automatically by preferring the trajectory with the higher cumulative sum of environment's ground truth reward value which is called scripted teacher method. Recently proposed preference-based reinforcement learning algorithms include exploration-driven data collection techniques that gather diverse state data [12] and transformer [13]-based preference predictors that alleviate manual labeling [14].

$$P_\psi\left[\sigma^1 \succ \sigma^0\right] = \frac{\exp\left(\sum_t \hat{r}_\psi\left(s_t^1, a_t^1\right)\right)}{\sum_{i\in\{0,1\}} \exp\left(\sum_t \hat{r}_\psi\left(s_t^i, a_t^i\right)\right)} \tag{1}$$

$$\mathcal{L}^{\text{Reward}} = -\mathbb{E}_{(\sigma^0,\sigma^1,y)\sim D}\Big[\, y(0)\log\big(P_\psi[\sigma^0 \succ \sigma^1]\big) + y(1)\log\big(P_\psi[\sigma^1 \succ \sigma^0]\big)\,\Big] \tag{2}$$

2.3. Graph neural network

A graph is a data structure consisting of nodes and edges connecting the nodes. Graph Neural Networks (GNN) [15] define node embeddings and edge embeddings for graph data and utilize update functions between nodes and their adjacent edges at each layer to learn node and edge embeddings.

An extension of the GNN methodology, the Graph Attention Network (GAT) [16], defines node embeddings as h_i and incorporates an attention mechanism to define the update function. GAT calculates attention scores between a node and its adjacent node embeddings, and the softmax value of the attention scores serves as the weight α_{ij}. These weights are then used to compute a weighted sum of adjacent node embeddings, allowing the model to update the node embeddings and capture the characteristics of each node (Eq. (3)).

$$h_i' = \sigma\left(\sum_{j\in\mathcal{N}_j} \alpha_{ij} W h_j\right) \tag{3}$$

3. Individual intrinsic reward functions using preference-based reinforcement learning

In cooperative multi-agent reinforcement learning with single global reward function, it becomes challenging to convey detailed information about individual agent policies. The proposed method, as described in Section 3.1, introduces a framework to learn individual intrinsic reward functions for each agent using preference-based reinforcement learning and Graph Attention Networks (GAT). Instead of using the global reward provided by the environment, the intrinsic reward functions are learned and utilized for training each agent's policy using DQN [17]. To validate the effectiveness of the intrinsic individual reward function, we train a single global reward using preference-based reinforcement

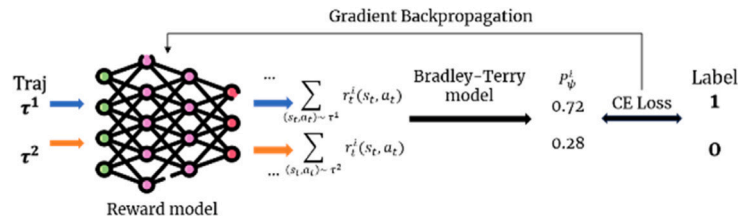


Fig. 2. Illustration of loss function and gradient backpropagation for individual reward function.

learning, and then evaluate the performance of policies trained with multi-agent reinforcement learning algorithms such as VDN [8] and QMIX [9] using the learned single global reward. Additionally, in Section 3.2 we introduce additional models that incorporate a graph-based method for learning intrinsic individual rewards, as well as models that do not, in order to assess the effectiveness of using the graph-based approach.

As shown in Fig. 2, each intrinsic individual reward model takes the observations and actions of agents as input and produces each agent's intrinsic reward as output. To model the relationship between an agent's reward r^i and the preference for that agent's trajectory segment σ^i, the preference probability P^i for each agent is defined in a manner similar to Eq. (1), using either the sum of individual rewards $\sum_t \hat{r}^i_\psi(o^i_t, a^i_t)$ or a weighted sum $\sum_t \hat{w}^i_\theta \hat{r}^i_\psi(o^i_t, a^i_t)$ with the Bradley–Terry model. Since the preference labels are provided as an overall team preference rather than individually for each agent, we assume that each agent's preference is identical to the team preference label, and we train the reward model using the cross-entropy loss defined in Eq. (2). In the remainder of this section, we describe the detailed structure of each proposed model.

3.1. Preference-based graph attention network

Preference-based Graph Attention Network (PBGAT) shown in Fig. 1(b) leverages the attention-based algorithm proposed in Graph Attention Networks (GAT) [16] to learn interactions between agents. It employs a graph embedding model to assign appropriate weights reflecting the contribution of each agent. In PBGAT, the individual reward value r^i_t is defined as the output of the node embedding of the Graph Attention Network. Each agent's trajectory $\sigma^i_{1:t}$ is encoded with each agent's trajectory encoder network $\hat{f}^i_\psi$. The graph network then receives each agent's encoded trajectory $\hat{f}^i_\psi(\sigma^i_{1:t})$ as input and calculates interaction weights using an attention mechanism, enabling the reward function to incorporate both individual observation and action information as well as interactions with other agents (Eq. (4)). Through this network architecture, the PBGAT model integratively learns inter-agent interactions via a graph structure, enabling each timestep's individual intrinsic reward to incorporate other agents' trajectories and capture information necessary for optimal policy learning.

$$r^i_t = \mathrm{GAT}_\psi(\hat{f}^1_\psi(\sigma^1_{1:t}), \ldots, \hat{f}^N_\psi(\sigma^N_{1:t})) \tag{4}$$

3.2. Additional individual intrinsic reward models

Preference-based independent Q-learning (PBIQL) shown in Fig. 1(a) is the simplest individual reward function model. Since each agent's reward function is inferred solely from its own observation and action information, without any information from other agents, this model does not take into account the relationships between agents. The individual reward r^i_t for each agent is defined by the output of its own reward network $\hat{r}^i_\psi$ using only each agent's trajectory data σ^i, with no dependence on other agents' trajectories (Eq. (5)). In PBIQL, the individual preference probability P^i_ψ is then defined based on the cumulative sum of each agents individual reward $\sum_t r^i_t$. PBIQL does not incorporate inter-agent relationships via graph structures or any other mechanism; accordingly, it serves as a baseline for comparison with the graph-based models presented later, highlighting the benefits of explicitly modeling agent interactions in multi-agent scenarios.

$$r^i_t = \hat{r}^i_\psi(\sigma^i_{1:t}) \tag{5}$$

Preference-based reward mixing network (PBRMIX) shown in Fig. 1(c) is a model designed to verify the weighting effect of the graph structure by learning both the individual rewards r^i_t and weights w^i_t during training, but using only individual rewards during inference. trajectory σ^i for each agent is encoded by the trajectory encoding network $\hat{f}^i_\psi$, and this encoding is input to both the reward network $\hat{r}^i_\psi$ and a weight network GAT_ψ employing a graph attention network. The reward network processes the encoded trajectory information through a simple MLP layer to learn individual reward values, while the weight network applies the GAT structure to the same trajectory encoding to learn each agent's interaction-aware weight w^i_t. In PBRMIX, individual preferences are modeled based on the weighted sum of rewards $\sum_t w^i_t r^i_t$.

$$r^i_t = \hat{r}^i_\psi(\hat{f}^i_\psi(\sigma^i_{1:t})) \tag{6}$$

$$w^i_t = \mathrm{GAT}_\psi(\hat{f}^1_\psi(\sigma^1_{1:t}), \ldots, \hat{f}^N_\psi(\sigma^N_{1:t})) \tag{7}$$

PBRMIX utilizes all agents' trajectories during training but calculates individual rewards during execution without relying on the graph structure. This structure may limit the ability of the reward function to fully capture interactions between agents.

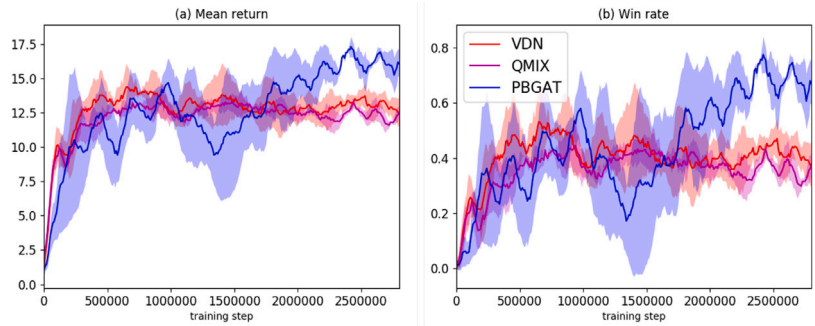
4. Experiments

4.1. Experimental setup and design

The experiments were conducted in the SMAC [18] environment, which is widely used in multi-agent reinforcement learning. The SMAC environment is based on the real-time strategy game StarCraft II and involves multiple agents cooperatively achieving the shared objective of eliminating enemy agents. Depending on the composition and number of allied and enemy agents, the environment provides symmetric or asymmetric scenarios, aiming to solve cooperative problems by learning individual control policies for each agent.

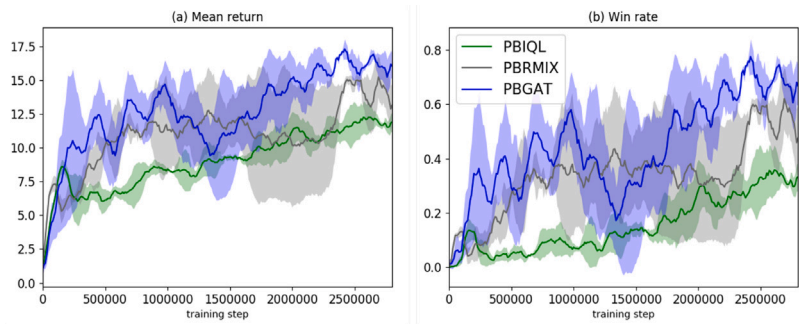
The SMAC environment provides both global state information and local observation information for each agent, making it suitable for testing and evaluating multi-agent reinforcement learning algorithms in Dec-POMDP settings. In this environment, we learned the individual reward functions proposed in Section 3 and used them to train each agent's policy solely with a simple single-agent reinforcement learning algorithm, DQN, without any additional algorithms, and then evaluated the performance. This demonstrates that the learned individual reward functions contain sufficiently detailed information required for each agent.

Baselines. To validate the effectiveness of the learned individual reward functions, we also implemented preference-based global reward functions described in Section 3 as baselines and applied them to the VDN and QMIX algorithms (hereafter referred to as VDN and QMIX, respectively). The results of these experiments are plotted alongside our proposed methods.



	VDN	QMIX	PBGAT
Max mean return	14.37±1.66	13.33±1.23	**17.3±0.73**
Max win rate(%)	53.22±13.59	44.47±9.8	**77.6±6.4**

Fig. 3. Results on the SMAC 3 m environment, showing mean ± standard deviation over four independent seeds. The red line is VDN, blue for PBGAT, purple for QMIX. (a) cumulative return, (b) win rate over training steps.



	PBIQL	PBRMIX	PBGAT
Max mean return	12.24±1.09	15.22±1.51	**17.3±0.73**
Max win rate(%)	35.72±9.27	62.08±11.23	**77.6±6.4**

Fig. 4. Results on the SMAC 3 m environment, showing mean ± standard deviation over four independent seeds. The blue line is PBGAT, gray for PBRMIX, green for PBIQL. (a) cumulative return, (b) win rate over training steps.

4.2. Experimental results

Fig. 3 compares the results of PBGAT, which is trained solely using individual reward functions, with those of VDN and QMIX, which are trained using a global reward function and multi-agent RL algorithms. Fig. 3 shows that the PBGAT model, which learns policies using individual reward functions as single-agent reinforcement learning, outperforms models that use global reward functions for policy learning in multi-agent reinforcement learning (VDN, QMIX). Based on cumulative rewards (maximum of 20), PBGAT achieves 17.3 compared to 14.37 for VDN and 13.33 for QMIX—an approximately 20% improvement. In terms of win rate, PBGAT attains 77%, while VDN and QMIX record 53% and 44%, respectively. These results highlight the challenges of learning a global reward function that provides sufficient information based on all agents' observations and actions—especially as the joint action space grows exponentially with the number of agents. Our PBGAT model leverages graph structure to learn individual reward functions that capture the information required for each agent's optimal policy, yielding superior performance in multi-agent scenarios.

4.3. Ablation studies and discussion

Fig. 4 compares the algorithms PBIQL, PBGAT, and PBRMIX, which are trained solely using individual reward functions. Fig. 4 indicates that PBIQL, which does not incorporate inter-agent interactions, achieves the lowest performance. Although PBRMIX learns interaction-based weights via a graph structure during training, its inability to apply these weights at test time leads to reduced performance relative to PBGAT. Based on cumulative rewards, PBGAT achieves a score of 17.3, while PBRMIX achieves a score of 15.2, indicating a 10% higher performance for PBGAT. In terms of win rates, PBGAT records a win rate of 77%, PBRMIX records 62%, and PBIQL fails to exceed 40%. While PBRMIX allows some interaction information to be learned by the individual encoding networks during the backpropagation process, it does not capture sufficient information.

These results demonstrate that using graph structures to appropriately learn agent interactions allows individual reward functions to capture optimal policy information effectively. Consequently, the individual reward functions learned by the PBGAT model not only maximize cumulative rewards provided by the environment but also improve win rates, effectively addressing the lack of information in global rewards by providing detailed information to individual agents.

5. Conclusion

This study introduces PBGAT, a novel graph-structured method for learning individual intrinsic rewards in multi-agent preference-based reinforcement learning that explicitly models inter-agent dependencies. By training each agent with a single-agent DQN on its learned individual reward, we show these rewards encode the policy-relevant information necessary for effective multi-agent learning, outperforming both global and independent reward model baselines. Future work will incorporate these individual rewards into full multi-agent architectures to enhance sample efficiency and resolve complex credit-assignment problems.

CRediT authorship contribution statement

Seokhun Ju: Writing – original draft, Software, Conceptualization. **Seungyub Han:** Writing – original draft, Formal analysis, Conceptualization. **Taehyun Cho:** Writing – original draft, Investigation. **Jungwoo Lee:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Taeyoung Lee:** Project administration. **Minkyoung Kim:** Project administration. **Jinho An:** Project administration.

Declaration of competing interest

The authors declare that there is no conflict of interest in this paper.

Acknowledgment

This work was supported by a grant-in-aid of HANHWA SYSTEMS, South Korea.

References

[1] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, vol. 17, 1999.
[2] D. Silver, et al., Mastering the game of go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.
[3] O. Vinyals, et al., Grandmaster level in starcraft ii using multi-agent reinforcement learning, Nature 575 (7782) (2019) 350–354.
[4] S. Kang, J. Yu, E. Seo, A reinforcement learning based adaptive container scheduling back-off scheme for reducing cold starts in faas platforms, J. KIISE 51 (3) (2024) 191–202.
[5] M. Pi, H. Lee, M. Chung, Reinforcement learning-based traffic signal control under real-world constraints, J. KIISE 48 (8) (2021) 871–877.
[6] S.W. Abeyruwan, et al., i-sim2real: Reinforcement learning of robotic policies in tight human–robot interaction loops, in: Conference on Robot Learning, PMLR, 2023.
[7] J. Foerster, et al., Counterfactual multi-agent policy gradients, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, 2018.
[8] P. Sunehag, et al., Value-decomposition networks for cooperative multi-agent learning based on team reward, in: Proceedings of the 17th International Conference on Autonomous Agents and Multi Agent Systems, 2018.
[9] T. Rashid, et al., Monotonic value function factorisation for deep multi-agent reinforcement learning, J. Mach. Learn. Res. 21 (178) (2020) 1–51.
[10] D.S. Bernstein, et al., The complexity of decentralized control of markov decision processes, Math. Oper. Res. 27 (4) (2002) 819–840.
[11] Y. Du, et al., Liir: Learning individual intrinsic reward in multi-agent reinforcement learning, in: Advances in Neural Information Processing Systems, vol. 32, 2019.
[12] K. Lee, et al., Pebble: Feedback-efficient interactive reinforcement learning via relabeling experience and unsupervised pre-training, in: ICML.
[13] A. Vaswani, et al., Attention is all you need, in: Advances in Neural Information Processing Systems, vol. 30, 2017.
[14] C. Kim, J. Park, J. Shin, H. Lee, P. Abbeel, K. Lee, Preference transformer: Modeling human preferences using transformers for rl, in: 11th International Conference on Learning Representations, ICLR 2023, 2023.
[15] M. Gori, G. Monfardini, F. Scarselli, A new model for learning in graph domains, in: Proceedings. 2005 IEEE International Joint Conference on Neural Networks, vol. 2, IEEE, 2005, pp. 729–734.
[16] P. Veličkovič, et al., Graph attention networks, in: International Conference on Learning Representations, 2018.
[17] V. Mnih, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.
[18] M. Samvelyan, et al., The starcraft multi-agent challenge, arXiv preprint arXiv:1902.04043.